

Lecture 02 — From Algorithm to Program

Algorithmic Thinking · Week 1, Session 2 — Friday 25 September 2026,
17:35–18:55 (lab 16:30–17:30 immediately before)

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview	1
Opener — a famous algorithm with no computer in it (10 min)	1
Notes	2
Stretch — if the session was easy	6
After the session	6

Objective. Translate a hand-written algorithm into a working Python program, and predict its output before running it.

Overview

On Tuesday you wrote algorithms and I broke them by reading them literally. Today the machine takes over that job, and it is far less forgiving than I was: where I paused and raised an eyebrow, Python simply stops and prints a word you have never seen.

So the shape of today is: your algorithm from Tuesday, typed in, **crashing**, and then fixed. Three failures, in order of how often they will happen to you this semester:

1. Everything the user types is **text**, even when it looks like a number.
2. Python has its own opinion about the order of operations, and it is not always yours.
3. A program that runs is not a program that is correct.

Opener — a famous algorithm with no computer in it (10 min)

Algorithms to Live By (Brian Christian & Tom Griffiths, 2016), chapter 1: **optimal stopping**. The setting: you are looking for an apartment in Querétaro in September. Every one you visit is gone by tomorrow; you can see at most ten; you must say yes on the spot or lose it. *When do you stop looking?*

Take three answers from the room before showing anything. “Take the first decent one” and “see all ten and pick the best” both come up. Both are algorithms. The second cannot even be executed — by the time you have seen ten, nine are gone.

The 37 % rule. Look at the first 37 % of the options and say no to all of them — but remember the best. After that, take the *first one that beats everything you have seen*. If none does, you get the last one. With ten apartments: look at three or four, then leap.

Strategy, ten options	P(you get the best)
Take the first one, no looking	10 %
Look at 1, then leap	28 %
Look at 3-4, then leap	40 %
Look at 5, then leap	37 %
Look at 8, then leap	19 %
See all ten, pick the best	impossible

With many options the best cut-off tends to 37 % ($= 1/e$) and so does the success rate. Two things to land: it is *provably the best possible strategy* — and it **fails 63 % of the time**. “Optimal” means *nothing does better*, not *always works*. That sentence returns in week 9, with data.

Then the question that ties it to Tuesday: *is this an algorithm?* Finite (ten visits), ordered, unambiguous (“beats everything seen” is a comparison, not a feeling), defined input (a sequence you can rank) and output (one apartment). It passes all four.

Your turn, 3 minutes, on paper: write the 37 % rule in Tuesday’s notation — INPUT:, OUTPUT:, numbered steps — for ten apartments seen one at a time. What exactly do you remember during the looking phase? What is the test that makes you say yes? Where does it stop — point at the step. *Keep the paper*. You cannot write this as a program yet: it needs the machine to **choose** (Tuesday) and to **repeat** (week 3). When you can, you will — this is the first program of week 3.

The same shape appears in hiring (the “secretary problem”, 1950s-60s), parking, selling a house, and — Kepler’s eleven candidates for a second wife — dating. Say the pivot out loud: *everything you just wrote is a decision or a repetition; today Python can only calculate*.

Notes

The anatomy of a program

Every Python program you write for the next ten weeks is made of these parts and nothing else:

Part	What it does	Example
Comment	A note for humans. Ignored by Python.	# calculate the speed
Multi-line comment	Several lines for humans — where your algorithm lives	"""INPUT: ... OUTPUT: ... 1. ..."""
Variable	A named box holding a value	distance = 100
Assignment	Puts a value in the box	speed = distance / hours
Expression	Something with a value	distance / hours
Input	Gets a value from the user (as text)	input("Distance: ")

Part	What it does	Example
Output	Shows a value	<code>print(speed)</code>
Function call	Uses code someone else wrote	<code>float(...), round(...)</code>

Gloss: *comentario* · *variable* · *asignación* — assignment · *expresión* · *entrada / salida* — input / output · *llamada a función* — function call

Every program in this course starts the same way: line 1 is your name in a # comment, and lines 2 onwards are your algorithm inside a """ multi-line comment — *then* the code. The quick exams take two points off for each of the two that is missing, so make it a habit today, not on the 29th.

Compare against Tuesday's pseudocode. READ became `input()`. `←` became `=`. DISPLAY became `print()`. **The structure is identical.** This is why we write the algorithm first: the translation is mechanical once the thinking is done.

Failure 1 — everything the user types is text

```
distance = input("Distance in km: ")    # "100"  <- text, not 100
hours = input("Time in hours: ")        # "2"    <- text, not 2
speed = distance / hours                 # TypeError
```

`input()` returns a `str` — a string of characters — always, without exception, even when every character is a digit. "100" and 100 are different things: one is a piece of text that looks numeric, the other is a quantity.

The fix is to convert:

```
distance_km = float(input("Distance in km: "))
hours = float(input("Time in hours: "))
speed = distance_km / hours                # 50.0
```

`float()` for anything that can have decimals, `int()` for whole numbers only.

From one measurement to an average. Tuesday's algorithm was called *average speed* and computed exactly one speed from one distance and one time — that is not an average of anything. A real trip is measured several times. With what we have today (no repetition yet) we can do it for a fixed number of measurements — three — by reading each pair, computing each speed, and averaging the speeds:

```
d1 = float(input("Distance 1 (km): ")); t1 = float(input("Time 1 (h): "))
d2 = float(input("Distance 2 (km): ")); t2 = float(input("Time 2 (h): "))
d3 = float(input("Distance 3 (km): ")); t3 = float(input("Time 3 (h): "))
v1 = d1 / t1
v2 = d2 / t2
v3 = d3 / t3
average = (v1 + v2 + v3) / 3
print("Average speed:", average, "km/h")
```

`07_average_speed_3.py`. Test: (100, 2), (90, 1.5), (140, 2) → 50, 60, 70 → **60.0**. Six input lines for three measurements: the program is *longer* than the algorithm, and it

is obvious that thirty measurements this way would be absurd. That is the itch week 3 scratches: *for N measurements — or a moving average over the last k — the program must repeat*. Note that it is not the same as $(d1 + d2 + d3) / (t1 + t2 + t3)$ — that is the overall average speed of the trip, a different quantity; both are correct answers to different questions, and the algorithm must say which one it computes.

The nastier version. Division crashes, so you find it immediately. + does not:

```
a = input("First number: ")    # "2"
b = input("Second number: ")   # "3"
print(a + b)                   # 23    <- not 5. No error. No warning.
```

On text, + means *join together*. This runs, produces a plausible-looking number, and is wrong. This is the failure mode you should actually fear: not the crash, but the silent wrong answer.

Failure 2 — precedence

Python evaluates in this order (highest first):

Operators		
1	()	parentheses
2	**	exponent — right to left
3	-x	negation
4	* / // %	left to right
5	+ -	left to right
6	< <= > >= == !=	comparison
7	not, and, or	logical

Three that catch everybody:

- $2 ** 3 ** 2$ is 512, not 64. ** groups right to left: $2 ** (3 ** 2)$.
- $-2 ** 2$ is -4, not 4. Exponent binds tighter than the minus sign: $-(2 ** 2)$.
- $10 / 3$ is 3.333... but $10 // 3$ is 3 and $10 \% 3$ is 1. Three different divisions. // is the quotient, % is the remainder — you will need % constantly from Week 3 for “is this number even?” and “is it divisible by?”

And the one that will actually cost somebody marks:

```
average = v1 + v2 + v3 / 3      # 133.33 for speeds 50, 60, 70
average = (v1 + v2 + v3) / 3    # 60.0    <- correct
```

That program runs. It prints a number. The number is nonsense, and nothing warns you. (The same trap with three grades 90, 80, 70 prints 193.33 instead of 80.0 — it is the single most common wrong line in this course’s history.)

Failure 3 — the program runs and is still wrong

This is the lesson underneath the other two. **Running is not a test.** The average program ran. The a + b program ran. Both were wrong.

The only defence available to you, today, is: **know what you expect before you press run**. If you predicted 60.0 and got 133.33, you have found a bug. If you had no prediction, you have found a number, and you will write it down.

This is also why we hand-check with known values. In 06_fuel_cost.py, 200 km at 100 km/h must be 2 hours. You know that without a computer. That is your test.

Built-in functions

You do not have to write everything. Python ships with functions for the common operations:

`abs()` `round()` `max()` `min()` `len()` `int()` `float()` `str()` `type()`

and, after `import math`: `math.sqrt()` `math.pi` `math.floor()` `math.ceil()` `math.pow()`

See examples/05_builtins.py. Note one thing carefully:

```
round(3.7)           # has a VALUE: 4. You can store it, add to it, pass it on.
print(3.7)           # has no value. It has an EFFECT: characters appear.
```

`x = round(3.7)` puts 4 in `x`. `x = print(3.7)` puts *nothing* in `x` — it prints 3.7 and stores None. Every semester several students write `total = print(a + b)` and cannot see why the next line fails. It is the same distinction as `return` versus `print`, which is next week's hard idea, and this is where it starts.

Gloss: *función predefinida* — built-in function · *valor de retorno* — return value
· *biblioteca / módulo* — library / module

Exercise 1 — Predict, then run (individual, 10 min)

Open examples/03_precedence.py. **Do not run it.** Write your prediction for all nine lines on paper first. Then run it and mark each one right or wrong.

Hand in the paper with your predictions — including the wrong ones. The wrong ones are the useful part, and they are not penalised.

Exercise 2 — Trip cost (plenary, built together, 20 min)

Write a program that, given a distance in km, a speed in km/h, the car's fuel economy in km/litre and the price per litre, displays the travel time, the litres required and the total cost in pesos.

Algorithm on paper first. INPUT: and OUTPUT: lines, numbered steps. I will look at the paper before anyone opens Thonny.

Then code it, together, from your own algorithm. Reference: examples/06_fuel_cost.py — but that is *my* translation of *my* algorithm; yours should translate yours.

Check with known values: 200 km at 100 km/h, 12 km/litre, 24.50 MXN/litre → 2.0 h, 16.67 litres, 408.33 MXN. If your program disagrees, your program is wrong. Save as A_matricula.py.

Exercise 3 — Celsius to Fahrenheit, in Trinket (*individual, 5 min in class, finish at home*)

$^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$. Read a temperature in Celsius, display it in Fahrenheit. **Write this one in Trinket** (<https://trinket.strivemath.org/python3>), not Thonny: it is the editor the quick exams use, and you should meet it before Tuesday, not during Tuesday.

Two required checks before you submit: 0 $^{\circ}\text{C}$ must give 32.0, and 100 $^{\circ}\text{C}$ must give 212.0. If you wrote `c * 9 / 5 + 32` without thinking about precedence, check whether you were lucky or right — and be able to say which.

Stretch — if the session was easy

Optional, ungraded, harder than they look. Bring them to asesorías or leave them in the week’s evidence box.

1. **Average without a memory.** Today’s program stored three speeds and averaged them. Suppose you could only keep *two* numbers: the average so far and how many measurements went into it. Show that `new_avg = old_avg + (x - old_avg) / n` gives the same result as adding everything up, then write the three-measurement program that way and test it with the same inputs. Which version would you trust with a million measurements, and why?
2. **Two averages that disagree.** `(v1 + v2 + v3) / 3` and `(d1 + d2 + d3) / (t1 + t2 + t3)` are both called “average speed”. Find distances and times where they differ by a factor of more than two. Then say, in one sentence each, which one a traffic officer wants and which one the driver wants.
3. **Predict, then run.** Write down, *before* running anything, what Python prints for `7 // 2`, `-7 // 2`, `7 % -2`, `2 ** 3 ** 2`, `0.1 + 0.2 == 0.3`, and `"3" * 3`. Run them. For every miss, write the rule you were missing — that rule is a line in someone’s quick exam.

After the session

Homework — assignments/hw-01-calculations.md. Due Sunday 27 September, 23:59, on Canvas. Two programs. The handwritten algorithm is part of the submission; code without it is incomplete.

Also before Tuesday:

1. **Respondus practice exam** — deadline Monday 28 September, 23:59.
2. **Adaptive levelling module** — lessons 1–5 (*Computer memory, Arithmetic expressions, Relational expressions, Logical expressions, Algorithms*), or whichever of them today’s diagnostic assigned you. The whole module (4%) closes **Thursday 8 October**; the final diagnostic (1%) is in the lab on **Friday 9 October**.
3. **Autoestudio** — relational and logical expressions, decision structures. We open Tuesday with your questions on it.
4. **Quick exam 1 is Tuesday 29 September**, first 20 minutes: algorithms, operator precedence, built-in functions, and programs requiring calculations.